Thinking Inside the Box: Injecting Realistic Radiation Faults in ML Accelerators

Bruno Loureiro Coelho†, Mani Sadati‡, Abraham Chan‡
Alex Hands*, Karthik Pattabiraman‡, Paolo Rech†
†*University of Trento, Italy*, ‡*University of British Columbia, Canada*, **TRIUMF, Canada*

***Abstract*—Machine Learning accelerators are increasingly deployed in safety-critical applications such as autonomous vehicles and space environments, where exposure to radiation poses serious reliability risks. To evaluate and mitigate this risk, we test tensor processing units (TPUs) with neutron, proton, and heavy-ion beams that account for 104 *million* years of natural exposure on the Earth, and more than 1,000 *years* of mission in outer space, showing that, contrasting with conventional single-bit models, faults affect localized rectangular regions of tensor outputs. We then build TPU-FI, an experimentally-tuned software-based fault-injector, predicting TPUs failure-in-time rates within, on average, 1.7x of beam results. Through 6 *million* software fault injections, we analyze how fault type, model architecture, layer type, and input data affect prediction correctness, finding that fully connected and convolutional layers exhibit higher vulnerability and different inputs trigger distinct fault propagation paths, especially in attention-based models.**

I. Introduction

Machine learning (ML) accelerators, such as Google's tensor processing unit (TPU) [1], AMD's AI Engine [2], [3], and NVIDIA's Tensor Cores [4], have become increasingly popular due to the high computational and power demands of modern ML workloads. These accelerators commonly employ *systolic arrays*, two-dimensional grids of processing elements optimized for multiply-and-accumulate operations, to significantly improve runtime efficiency. As these accelerators are being deployed in safety-critical domains such as autonomous vehicles and space robotics [2], there are growing concerns about their reliability to radiation: silent data corruptions (SDCs) [5], [6] are particularly critical, as they may result in misclassifications, thus leading to unsafe behavior.

The current standard for qualifying the radiation reliability of ML accelerators is to conduct beam experiments using real hardware [7]–[10]. However, each target radioactive environment requires a dedicated qualification campaign: testing with atmospheric neutrons may suffice for terrestrial applications, but does not capture the effects of proton radiation in low-earth orbit, or heavy-ion exposure in deep-space missions. These beam experiments are not only expensive and time-consuming, but also only offer limited visibility into how faults originate and propagate. This is especially problematic for systolic arrays, which are monolithic units with limited internal observability; faults can only be inferred from the final model output, offering no insight into where they originated or how they propagated [11], [12]. This is in sharp contrast to CPU/GPU-based platforms, where faults can often be traced through separated compute and memory pipelines [10], [13], [14]. Beam experiments provide realistic measurements, but are impractical for design-time analysis and large-scale testing.

Software-based fault-injection (FI), in contrast, is flexible and allows large-scale experiments, but existing approaches suffer from major limitations: they rely on synthetic fault models, such as random single- or multiple-bit flips [15]–[17], which do not reflect the spatial and statistical patterns observed in real radiation-induced faults (as shown later). Furthermore, they ignore important hardware effects, such as fault clustering and parallel data movement, leading to unrealistic results. Alternatively, register transfer level (RTL) or micro-architectural FI [14], [18], [19] offer greater fidelity, but are prohibitively slow (even for simple devices), and require detailed knowledge of proprietary ML accelerator architectures, which is rarely available [10], [11], [20]. *As a result, neither beam experiments nor FI provides a scalable, accurate, and practical way to assess the resilience of ML accelerators to radiation.*

In this work, we first perform extensive radiation beam experiments on Google's Coral Edge TPU, capturing realistic effects from terrestrial, orbital, and deep space-grade radiation (neutrons, protons, heavy ions)[1]. *These experiments reveal that faults often affect contiguous rectangular regions in output tensors (i.e., a box), rather than result in a single bit-flip.* Based on this insight, we design TPU-FI[2], a software-level FI methodology, by modifying TensorFlow's[3] kernel implementations to accurately emulate such "box-style" faults. TPU-FI is a software-based FI framework that enables scalable and realistic evaluation of ML application reliability. TPU-FI not only estimates the failure-in-time (FIT) rate of ML models (FIT rate is the number of failures in a billion hours of operation), but also enables in-depth analysis of how injected faults propagate through the model and lead to misclassifications.

To the best of our knowledge, TPU-FI is the first software-level FI framework that accurately replicates the fault behavior observed with beam experiments for ML accelerators.

In summary, we present the following contributions:

- We conduct accelerated beam experiments that account for over 104,000,000 years of natural radiation exposure on Earth and over 1,000 years of space exposure;
- From our experimental data, we derive a **realistic, hardware-driven fault model** for the Coral Edge TPU,

[1]Data and code artifacts are available at zenodo.org/records/19202320.
[2]TPU-FI is open-source: github.com/DependableSystemsLab/TPU-FI.
[3]TPU-FI can be easily extended and ported to other ML frameworks.

that is valid for both terrestrial and space applications;
- We implement **TPU-FI**, a software-based FI that incorporates the above fault model into TensorFlow;
- We use TPU-FI to **estimate FIT rates of ML models** within $1.7\times$ of the measured beam results, significantly outperforming all prior reliability frameworks in both accuracy and performance. We have also found that traditional software-based FI methods that use a uniform single-bit flip fault model under-report up to $\approx 4\times$ fewer misclassifications compared to our approach;
- Using TPU-FI, we study how model architecture, layer type and position, and input characteristics affect reliability in **five diverse ML models**, such as convolutional neural networks (CNNs) and vision transformers (ViTs).

Our analysis reveals four key insights: (1) **fault resilience** is highly **dependent on layer type** - fully connected and convolutional layers are particularly vulnerable, while transpose convolutions consistently show negligible impact. (2) Within the same architecture family, **deeper and wider models** tend to be **more vulnerable to faults**, presenting up to $5\times$ higher FIT rates. (3) **Fault propagation paths vary with input data**, that is, some inputs activate specific attention heads and pathways, leading to structured and input-dependent fault sensitivity. Finally, (4) **misclassifications** are highly **input-dependent**, with some inputs presenting no misclassifications, while other inputs may have up to 70% of misclassifications.

These results reveal important aspects to consider when designing techniques to improve the resilience of ML models. Furthermore, because TPU-FI is based on a realistic fault model, the results of FI campaigns more accurately depict the effects of real radiation-induced errors, thus making TPU-FI indispensable to evaluate fault-tolerance techniques.

II. Background

A. ML accelerators in safety-critical applications

Processing complex data is crucial in many safety-critical applications, such as autonomous vehicles, unmanned aerial vehicles (UAVs), and on-board processing in satellites [10]. The State-of-the-Art is dominated by ML models, with breakthrough performance first being achieved by CNNs such as ResNet [21], and, more recently, with the ViT architecture - extremely large ML models that process the *entire image* at once [22]. ViTs include up to several hundred million parameters ($25\times$ more than a ResNet CNN), thus requiring considerable processing power to run. Fortunately, ML models can be efficiently mapped to hardware accelerators based on systolic arrays, such as Tensor Cores in GPUs [10], AI engines in AMD Versal ACAPs [3], [8], or Google's TPUs [1].

Systolic arrays move data from input buffers towards output accumulators, multiplying input data and model weights at each time step [23]. This deep pipeline of multiply-add operations can significantly speed-up feed-forward neural networks, such as ViTs, and CNNs [1]. Due to its high efficiency, and the rising popularity of ML models, the systolic array architecture is being adopted by most hardware accelerators in industry.

Google's Coral Edge TPU is a highly energy- and cost-efficient alternative to hardware accelerators such as GPUs [1], [10]. With a power draw of only 2W, the Coral TPU has a maximum computing power of 4 Tera operations per second (TOPS), resulting in a power-efficient 2 TOPS per watt [1], [10], [23]. The Edge TPU is largely a functional unit (FU), with simple drivers in a small control unit, and a large *systolic array* responsible for arithmetic operations over input data [10]. While this work utilizes the Coral Edge TPU as a case study, the methodology proposed and results obtained are easily extensible to ML accelerators based on systolic arrays.

B. Reliability evaluation of ML accelerators

Commercial-off-the-shelf (COTS) ML accelerators offer incredible flexibility and efficiency [1]. However, since their reliability is not certified, these devices must be thoroughly evaluated before deployment in safety-critical applications, especially in harsh environments with high levels of radiation, such as in space applications [9], and in aviation, where the higher altitude exposes aircrafts to high levels of radiation, leading to potentially harmful operational errors as recently published in the news [24]. We focus on the effects of silent data corruption (SDC), *which are any deviation in the output of the models*. We further categorize these events into (a) **tolerable SDCs**, when the ML model outputs slightly incorrect probabilities (or values), but the final *classification is unaffected*, and (b) **critical SDCs**, where the *final classification* has been *changed* due to radiation-induced errors. We do *not* report detected unrecoverable errors (DUEs) since they are very rare in systolic arrays (1-2 orders of magnitude lower than SDCs [9], [10]), due to the lack of a scheduler and operating system. They are also less critical, since they are detected.

The evaluation of the system reliability can normally be performed by three methods [10], namely (i) *accelerated beam experiments*; (ii) *low-level FI*, such as circuit/gate simulations, RTL and microarchitecture-level FIs; (iii) *high-level FI*, such as software-level injection. We summarize them below.

Accelerated beam experiments produce realistic data, since faults are generated by the interaction of particles with Silicon devices. Unfortunately, they have two limitations, (1) the cost and complexity of the experiment, that should be repeated for each new model and each radiation source, and (2) limited visibility into how the initial fault propagates through the software stack [11], [12]. Given that the observable failure propagates through a series of complex hardware and software mechanisms, the origin of the fault is difficult to be determined, meaning that we cannot pinpoint the most vulnerable (software or hardware) components of our system [10].

Low-level FI at micro-architectural or register-transfer levels (RTL) allows full visibility into how faults propagate through the system, and potentially lead to errors [10]. Low-level FIs have been successfully utilized with CPUs [14], [18], and GPUs [19], providing useful insights, and even open-source software to study the reliability of these devices [6]. However, for proprietary ML accelerators, low-level information is not available, the RTL description is proprietary, and

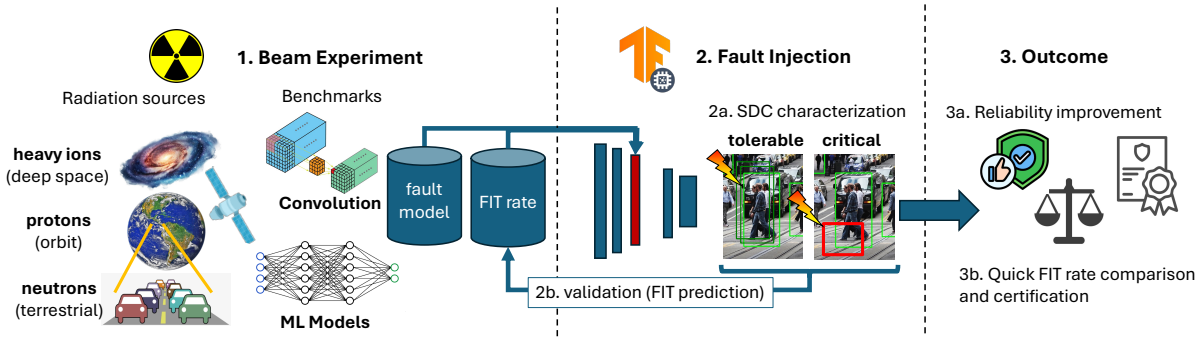


Fig. 1. (1) The realistic fault model in TPU-FI is based on the results of accelerated beam experiments, considering terrestrial, orbital, and deep space environments. We test both atomic convolutions and complex ML models. (2) We perform FI to characterize SDCs and to validate FIT prediction comparing the results of TPU-FI with beam data. (3) We use our insights to improve the reliability of the system, and ease FIT rate comparison and certification.

the documentation is sparse, limiting micro-architectural FI to behavioral models. Therefore, *low-level FI is simply not possible*. Even if it was, RTL and micro-architectural FI time increases exponentially with the complexity of the device, meaning that it would take an exceedingly long time (order of *years* [12]) to simulate highly complex accelerators.

Software FI is the most time-efficient approach, as it involves modifying some (architecturally) visible values, such as the intermediate output tensors of ML models during inference. Due to its efficiency and (relative) simplicity, this approach has been extensively explored [15]–[17], and has been found to be accurate for CPUs and GPUs [25]. However, the traditional single- or double-bit flip fault model is *not realistic* for ML accelerators. In fact, the original radiation fault propagating through a systolic array is unlikely to result in a single bit flip in one element of the tensor. We find that (Section V-B) only about 50% of observed convolution corrupted outputs have just a single wrong element.

Cross-layer hybrid FI approaches have been applied to study the reliability of CPUs and GPUs [11], [12], combining accurate low-level injections (such as micro-architectural or RTL FI, when available) with more flexible software-level injections, achieving satisfactory scalability while preserving accuracy [12], [26], [27]. A few recent efforts have extended hybrid FI to ML accelerators, combining RTL FI with software-level analysis. However, these approaches typically rely on simplified fault models, such as isolated single-bit flips or dense tensor-wide corruptions [28]–[30] and, crucially, they are *not validated on real hardware under radiation*. As we show in Section IV, beam-based fault models reveal structured corruption patterns (e.g., sparse rectangular regions) that differ significantly from these assumptions. Our approach combines *software-level FI* with data gathered via accelerated *beam experiments*, providing fast results with low costs, while also ensuring full visibility and realistic results.

III. Main Idea and Contribution

We propose a cross-layer methodology to implement, in the software level, a realistic fault model obtained experimentally from extensive beam testing. Our methodology has three steps, as illustrated in Figure 1. In the first step, we performed **accelerated beam experiments** to gather realistic SDC data on large convolutions running on the Coral TPU exposed to neutrons, protons, and heavy ions. Because convolutions are the target operation of systolic arrays, we consider a convolution as the operation with the finest/atomic granularity. Due to the difficulties in simulating or modeling systolic arrays on a register or processing-element level, we use the beam data to **model how faults affect the outputs of convolutions**. Using the experimental data collected, we then formalize a realistic fault model for software-level injections: the output of each layer is modified according to a distribution, ranging from a single element, to multiple elements arranged in varying ways, as observed in our accelerated beam experiments (Section IV).

This fault model obtained above is used to (*step 2* in Figure 1) **perform FIs** using a software-level tool that we developed, TPU-FI. We inject faults on a *per layer* basis, for each fault type (we consider three different fault types - Section IV), for each input image: 100 injections for each combination, or 3200 injections for each of the three fault types in each layer, resulting in 9600 injections per layer. During each injection, (*Step 2a*) we track how the initial fault propagates through the model, in order to investigate the most vulnerable layers of each model, and compute the probability for each layer to propagate a fault all the way to the output, i.e., if we inject a fault in layer i, what is the probability that it changes the *final output* of the model?

As a result, (*step 3* in Figure 1) by **understanding** the intricate **relationship between the layers**, and how different fault types affect the model, we can design software-based solutions to improve the reliability of machine learning models. These insights would not have been possible with beam experiments alone, due to the lack of visibility, and would have been highly inaccurate with FI using a synthetic fault model such as single bit-flip faults (as we show in Section VII-B).

Finally, by precisely predicting the FIT rate of ML models using TPU-FI (*Step 2b*), **we obviate the necessity to test new models with costly and time consuming beam experiments**. *Our tool estimates the FIT rate of complex ML models within, on average, 1.7× of beam data, vastly outperforming existing*

methods that provide, when applied to simple codes and devices, differences on the order of 10× [11] (Section VI-B).

IV. Benchmarks and ML models tested

To evaluate our cross-layer methodology, we use both beam experiments and FI on three different ML models with varying configurations, resulting in a total of five models.

- **ViT**: We use an implementation of the base Vision Transformer [22] adapted to the Coral Edge TPU. We train a ViT for image classification, with 16 attention heads, and 6 transformer layers.
- **U-Net + MobileNetv2**: The U-Net architecture [31] offers accurate semantic segmentation (each *pixel* of the image receives a *classification*), and is a very efficient architecture during training. Additionally, the *encoder* (contracting path), responsible for extracting the features from the image, can be a pre-trained classification model, such as a ResNet [21], or a MobileNetv2 [32] CNN.
- **DeepLabv3 + MobileNetv2**: DeepLabv3 [33] is an architecture for semantic segmentation models that offers state-of-the-art accuracy by applying *atrous convolution* layers. Similarly to U-Net, the DeepLabv3 architecture includes an *encoder*, which can be a pre-trained classification CNN, such as MobileNetv2 [32].

The selected models are the most advanced and complex ML applications that can be executed on the Edge TPU. They include a state-of-the-art ViT model and two popular semantic segmentation CNNs with differing architectures (U-Net and DeepLabv3). The latter two models are exemplary use case for systolic arrays, since the multiply-add architecture is optimized for feed-forward networks. Moreover, semantic segmentation is the most computing demanding task in image processing, as the model must classify each pixel of the input image.

The adapted ViT model was chosen as it pushes the limits of the Edge TPU. Unlike CNNs, transformer models require several adaptations to be ported to the Edge TPU, as the device does not support many of the specialized layers, nor the intricate activation functions commonly found in state-of-the-art transformers [34]. Additionally, implementing transformers in the Coral Edge TPU often limits the size of the model due to the relatively small buffer for model weights [34]. We used several datasets, including CityScapes [35], OxfordPets [36], CIFAR100 [37], and VOC [38]. For each dataset, we select inputs with a mix of high- and low-confidence inputs, as well as a common and balanced set of classes.

V. Experimentally-tuned Fault Model via Beam

This section describes the methodology employed, and the results obtained. Leveraging these results, we formalize a fault model for our software injector (described in the next section).

A. Accelerated beam experiment setup

Achieving high accuracy when evaluating the reliability of a system requires a realistic fault model to propagate, at the software-level (as mentioned in Section III), faults that accurately represent radiation-induced errors that happen in the natural environment. To this end, the first step of our strategy is the gathering of representative data through extensive accelerated beam experiments using multiple testing facilities. Specifically, we perform experiments with (1) **neutrons (terrestrial applications)** at TRIUMF, Canada, and ChipIR, UK, two facilities widely used to measure FIT rates for terrestrial applications, both compliant with the JEDEC standard [39], and to qualify electronic devices reliability to the ISO26262 standard [40]; (2) **protons (low-orbit deployments)** at both CNAO and TIFPA, Italy, both of which are medical facilities delivering protons with energies similar to the protons found in low-orbits (200MeV), according to the reliability test standard of the European Space Agency (ESA) [41]; and (3) **heavy-ions (for deep-space scenarios)** at RADEF, Finland, and UCL, Belgium, which are facilities designed by ESA to certify the FIT rates of electronic devices in deep-space missions [41].

We present data from over 430 hours of *beam time*, with a total fluence (particle count) of more than $10^{13} particles/cm^2$ - our experiment is equivalent to roughly 104 *million years* of natural exposure on Earth (13 $neutrons/cm^2/h$ [39]), and more than 1,000 *years* of mission in outer space. Despite the high acceleration factor during experiments, we tune the flux so to make negligible the probability for two different particles generating faults in a single execution (error rate lower than 1/100 executions). This means that nearly *all observed multiple errors are caused by the same particle*.

Overall, our experimental data includes over 13,000 observed SDCs, providing statistically significant results with several models, and deployment scenarios. While there are minor differences in the physical setups for our experiments (such as the cable length, and the mounting of the device, as shown in Figure 2), we use *exactly the same* scripts and host device. Furthermore, we execute *identical* codes with *the same* inputs in all facilities to reduce the experimental uncertainty.

To increase statistical confidence, we irradiate 2 TPUs with protons, 4 TPUs with neutrons, and 2 TPUs with heavy ions. Each TPU was only irradiated with one source, that is, TPUs used with neutrons were not re-used for protons or heavy ions. As shown in Figure 2, the TPUs are aligned in front of the radiation source, with the beam covering the entire chip area. Each TPU is connected via USB to a host device (Raspberry Pi 4) which is not irradiated (not shown in the figure).

When performing accelerated beam experiments, each TPU is connected via USB to a host (not irradiated) responsible for the control logic of the program, whereas the TPUs are exclusively used to run the benchmarks. During the test, each host device first loads the model, input, and pre-computed golden output files to memory. The model is, then, loaded into the TPU via Coral's TensorFlow API, generating an *interpreter*. The interpreter is called to interact with the TPU, including loading input tensors for processing, invoking the model (starting inference), and retrieving the results once processing is completed. Once the inference results are available, the host device compares the computed tensor to the pre-computed golden values: if there is a mismatch, the differing values are logged for posterior analysis, and an SDC is recorded. Finally,

we clear the device caches and reload everything from storage (model, input, golden files) to avoid cumulative errors.

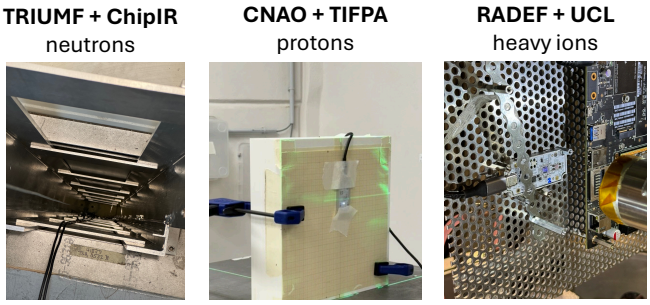


Fig. 2. Physical setup for beam experiments with the Google Coral TPU.

To measure the error rate of the device, we divide the number of observed events (SDCs) by the *fluence* of particles received by the device ($particles/cm^2$), which can be computed with the particle counts provided by the facility. As described in the JEDEC standard for radiation testing [39], the fluence in our calculation takes into account only the particles hitting the device during benchmark execution, discarding the fluence received during setup, loading, or downloading of data (as during these steps, the TPU is idle). For charged particles (protons, heavy ions), our setup also includes an automatic control for the beam shutter that blocks particles when the device is not executing the benchmarks. This prevents the device from receiving unnecessary *total ionizing dose* [42], which can lead to permanent failures or malfunctions. During our experiments, we observed neither any dose-related events nor cumulative effects. As previous work has found [9], [10], the TPU DUE rate is 1-2 orders-of-magnitude lower than SDC rate. Thus, we only consider SDCs in our evaluation.

B. Fault Model and Distribution

Besides measuring the error rates of ML models, beam experiments also provide a realistic fault model for operations in systolic arrays. By exposing the hardware to radiation, we can observe realistic errors at the final output of the ML model or micro-benchmark. Our idea is to characterize the fault model of the hardware accelerator by executing its atomic operation (convolution), so that we can incorporate the observed SDCs into a fault model for the software FI. We choose large input matrices for the convolutions to fully utilize the hardware ($1024\times1024\times3$ input/output elements). Then, we download all of the output data affected by radiation, subsequently categorizing the logged SDCs.

Based on the experimental data, we find that nearly half of the radiation-induced errors affect only a **single element** in the output, meaning that only *one* position of the output matrix has a value that differs from the expected one. Note, however, that this does *not* mean that single FI is realistic 50% of the times: single element errors are considerably less likely to propagate to the output of the ML model (Section VI-A)

We also observed that, while most errors *are* single-bit flips (with several output elements having a bit flipped), half of the errors happen in the least significant bit, while the other bits have similar probabilities of being affected, i.e., when looking at the *output value* of a layer affected by radiation, the quantized integer element only differs by 1. This is because the Coral Edge TPU operates over uint8 data: after multiplying an 8-bit value by an 8-bit weight, the partial products are summed in the accumulator, necessarily in 16 bits or more. After all of the partial products are computed and accumulated, the final value is quantized back to a single 8-bit value. Thus, while the 8 bits of the input or weight have equal probability to be corrupted, this no longer holds for the result: a small difference in one of the partial products (e.g., differing by 100) only changes the final summation (before quantization) by a small proportion (e.g., only 2% difference from the expected value).

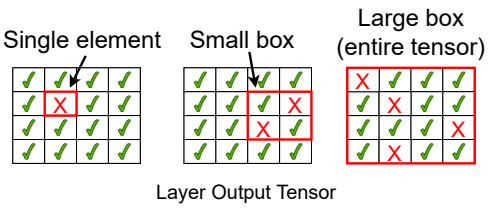


Fig. 3. We characterize the error based on the size of a "bounding box" that includes every incorrect element.

Particularly, if we consider a 16-bit accumulator, there are $256\times$ more possible values than in the 8-bit result, meaning that the difference in the accumulator must be, in the worst case, ≥256 to modify the quantized output. In practice, because each output depends on the result of multiple (16-bit) multiply-add operations (while the exact number is unknown, some estimates place it at around 64×64 processing elements), the accumulator is likely implemented with 24 to 32 bits. This means that, once this final value is quantized back to 8 bits, the small difference (e.g., 2%) results in a quantized value that is similar to the expected error-free value - rather than X, the erroneous output is quantized as X$\pm$1, hiding the larger difference resulting from a bit-flip in a random bit. Therefore, it is more likely for a fault during computation to lead to erroneous output that only differs by ±1.

The remaining errors show **multiple elements** affected by the initial fault (that is, several positions in the output vector differ from the expected fault-free values). This is in accordance with studies on similar devices and general parallel architectures [10], [43]. In addition to the number of affected elements, as shown in Figure 3, we analyzed how the elements are *distributed*, that is, whether they are concentrated in a small subregion of the 2D output tensor, or if they are scattered across the entire tensor area.

Figure 4 shows the cumulative density function (CDF) of the *size* of the *bounding box* ($height\times width$) that includes every incorrect element in the output tensor. While a bounding box includes every element that was affected by the fault, it does **not** mean that *every element* in the box is affected. According to these results, nearly 50% of the errors observed only have a single element affected, while roughly 30% of multiple

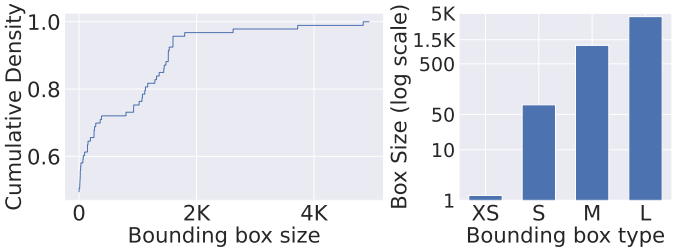


Fig. 4. Distribution for the size of the bounding boxes observed during beam experiments. (a) CDF (left), and (b) size per box type (right).

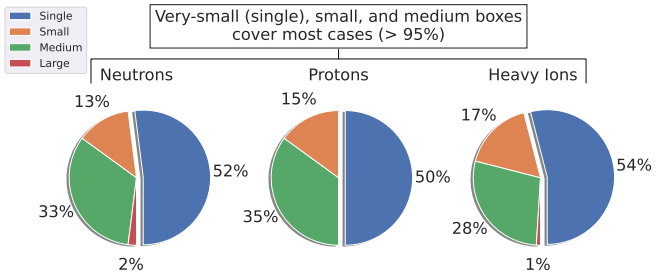


Fig. 5. Error distributions for convolutions observed for different particle types in beam experiments. Results are similar for the three particle types.

element errors are concentrated in boxes of sizes between 5 and 300. The remaining 20% of the errors have varying bounding box sizes, ranging from 500 to 1,800 elements, with a few outlier errors spread out up to 5,000 elements apart.

Additionally, despite the low density of incorrect elements in the bounding box (that is, not many elements are incorrect, and the incorrect elements are very far apart), our experimental methodology ensures that these errors are still caused by a ***single particle***. Thus, we believe that the causes for these errors are similar to how multiple-element errors are reported in the literature, where parallel devices such as GPUs can present multiple errors due to a bit-flip in either an instruction or model weight value [19], [44]. However, unlike in those devices that use floating-point operations, the TPU uses quantized uint8 values, and, as previously explained, the accumulators in systolic array must re-quantize the final sum back into an uint8 value. We posit that, while multiple-element errors originate from a single model weight value being corrupted (which in turn affects the computed sums for *every* output element[4]), these computed sums have similar numerical values to the expected sums (error-free). In turn, once each of these values is re-quantized back to uint8, the result in the output tensor is the same as the fault-free golden value (despite the values differing before quantization). Thus, while the resulting error affects multiple elements in the output, the quantization masks the effect in several elements, resulting in bounding boxes with the observed low density values.

Observation I

Only half of the SDCs affect a single element in the model output, whereas the other half follow distributions for size and density of corrupted tensors. Multiple-element errors can be partially masked by quantization, resulting in sparse rectangles containing the erroneous output elements.

Aiming to provide a realistic formalization that can be used for software FI, we consider that, when injecting faults in the *output tensor* of a layer, usual sizes for ViTs (described in Section IV) are 256, or 4096 elements in the layer output, while segmentation can vary between 256 and 65536 elements, with a median size of 16384. Therefore, while the data collected during beam experiments includes other bounding box sizes, we only consider single, "small", and "medium" fault types due to the low probability of other types (*these three fault types represent over* 95% *of all errors observed*), and the fact that other box sizes are generally larger than the output of the layers in ML models. In other words, when scaling down large boxes to the size of the output tensor, the reduced size and density will approximate medium boxes, thereby having an overall negligible impact on the final results.

Observation II

Over 95% of SDCs can be described by single element errors, "small-sized", or "medium-sized" bounding boxes.

Thus, aiming to provide a realistic fault model, while also allowing a simple and efficient implementation, based on these experimental results, we categorize errors into bounding boxes ranging from small to large sizes, as shown in Figure 4. Finally, the results in Figure 5 show that there are only minor difference in the fault distributions obtained by neutrons, protons, and heavy ions.

Considering these results, we devise the following fault model, as shown in Table I: 48% of faults affect a single element, while the remaining faults consist of "small-", and "medium-sized" bounding boxes. Although the medium boxes have a lower density (3.5%) than small boxes, due to the larger size, they affect significantly more elements in the output tensor of a layer. Finally, as previously mentioned, the distribution for bit-flips is *not uniform*, with the **least significant bit** being affected 59% of the time by the fault. We reflect this in our fault injection experiments as well.

TABLE I
FAULT INJECTION PARAMETERS BY FAULT TYPE

Parameter	Single-pt	Small-box	Medium-box
Fault Type Probability $\mathcal{P}(f)$	48%	15%	37%
Area Range $\mathcal{A}_\beta(f)$	$[1, 1]$	$[41, 113]$	$[949, 1351]$
Error Density $p_{\text{inj}}(f)$	100%	7%	3.5%
Bit-Flip Distrib. $\mathcal{D}_{\text{bit}}$	$\Pr(b = 0) = 59\%,\ \Pr(b = 1 \ldots 7) \approx 5.9\%$		

[4]In order to compute a single output value in a convolution operation, a subset of input values are multiplied by the entire convolution kernel, meaning that an error in a given kernel value affects *every* output element.

VI. TPU-FI: Software-Level Fault Injection Tool

In this section, we describe TPU-FI, our FI framework, and how we use it to estimate the FIT rate of ML models.

A. Fault Injection Framework

We develop TPU-FI, a software-level FI framework for quantized TensorFlow Lite models that mimics the observations from beam experiments. Rather than injecting faults at the micro-architectural level, which is impractical or inaccurate for ML accelerators, our framework injects faults directly into the intermediate outputs of ML layers during inference to mimic the effects of radiation-induced faults in Section IV. This allows observing how faults propagate through subsequent layers of the model, and accurately predicting the FIT rate of the model executing on TPUs.

Critically, the injected faults are not limited to single bit-flips. Instead, they are derived from the hardware fault model obtained through beam experiments (Section V-B), capturing realistic error patterns such as clustered bit flips over rectangular regions of the tensor. The methodology for the FI is shown in Algorithm 1, and consists of the following steps:

Algorithm 1: Fault Injection in TPU-FI

Require: Layer L, fault type f

1: $s \sim \mathcal{U}_{\mathbb{Z}}(\mathcal{A}_{\mathcal{B}}(f))$ *// Sample box size*
2: $\mathcal{R} \sim \mathcal{U}(\mathcal{S}_{\text{rect}}(L_{\text{out}}, s))$ *// Sample output region*
3: **for** each element $e \in \mathcal{R}$ **do**
4: $z \sim \text{Bernoulli}(p_{\text{inj}}(f))$
5: **if** $z = 1$ **then**
6: $b \sim \mathcal{D}_{\text{bit}}$ *// Sample bit index*
7: $e \leftarrow \text{FlipBit}(e, b)$ *// Inject fault at bit b*
8: **end if**
9: **end for**

1) The user specifies a target layer L and a fault type $f \in \{\texttt{single}, \texttt{small-box}, \texttt{medium-box}\}$. For each (L, f) pair, we run 3,200 FIs (100 per image over 32 input images).
2) A fault region size s is sampled from the fault-type-specific size distribution $\mathcal{A}_{\mathcal{B}}(f)$.
3) A rectangular region $\mathcal{R}$ of area s is uniformly sampled from the layer output tensor L_{out}.
4) Each element $e \in \mathcal{R}$ is selected for injection with probability $p_{\text{inj}}(f)$.
5) For each selected element, a bit index $b \in \{0, \ldots, 7\}$ is sampled from the empirical bit-flip distribution $\mathcal{D}_{\text{bit}}$, and bit b of e is flipped.

Example: Suppose we want to inject a fault into layer 16, a fully connected layer with a 500×500 output, using the `small-box` fault type. During inference, we modify the output immediately after it is produced by layer 16. First, we sample the box size $s \sim \mathcal{U}_{\mathbb{Z}}(\mathcal{A}_{\mathcal{B}}(\text{small-box}))$; assume $s = 100$. Then, we sample $\mathcal{R}$, a random region of the output tensor with size 100 ($\mathcal{R} \sim \mathcal{U}(\mathcal{S}_{\text{rect}}(500 \times 500, 100))$); Assume $\mathcal{R}$ is a 10×10 sub-matrix out_L`[200:210, 300:310]`. For each of the 100 elements in $\mathcal{R}$, we inject the fault with probability $p_{\text{inj}}(\text{small-box}) = 7\%$. That is, for each location, the only two possible outcomes are to either inject or not to inject (a Bernoulli distribution) [45]. If selected, we sample a bit index $b \sim \mathcal{D}_{\text{bit}}$; e.g., $b = 5$, and flip the 5th bit of that element.

After modifying the selected elements, the model resumes the inference. If the final output differs from that of the fault-free baseline, an SDC is recorded. Moreover, if the prediction is incorrect or the segmentation mask is significantly affected (at least 1% of the pixels), we record it as a critical SDC.

B. FIT rate estimation

Combining FI using TPU-FI and realistic data gathered from accelerated beam experiments, we can accurately estimate the FIT rates of ML models. This estimate is based on two measurements: (1) the probability for a fault to occur in one layer, and (2) the probability for a fault to propagate to the output of the model (that is, the probability to cause an SDC). While the latter is directly measured via FI, the probability for a fault to occur in a given layer depends on both the radiation sensitivity of the hardware (measured with beam experiments), and computational complexity of the layer (i.e., how much of the hardware is being utilized by the model).

To scale the beam experiment data to the FI prediction, we make four assumptions as follows:

1) At most one layer is assigned to the ML accelerator: Because feed-forward NNs have sequential layers, the current layer must be completed before processing the next layer.

2) At most one fault can occur in a layer: the relatively low natural particle flux (tens of $neutrons/cm^2/hour$ on the Earth's surface, and a few $ions/cm^2/second$ in outer space [39]), combined with the measured probability for the impinging particle to corrupt the hardware (Section IV), ensures that at most one particle can induce a fault during the execution of a ML model. This is also maintained when performing accelerated beam experiments (Section IV). *Multiple* observed errors are thus caused by a *single* particle.

3) The hardware FIT rate saturates: if the size of a layer exceeds the hardware resources, i.e., as shown in Equation 1, if it requires more operations than the device can perform in parallel, the scale factor S_{layer} (used to multiply the FIT rate in the next step) saturates to the maximum hardware utilization. Under a constant flux of particles, using the same twice hardware doubles the observed error number (double operations are executed) but also doubles the number of impinging particles, since computation takes longer [10]. Given that the FIT *rate* is calculated as the number of errors divided by the number of particles, the resulting value is the same.

4) Fault types are independent events: each of the fault types identified in Section V-B (single-point, small-box, and medium-box) are statistically independent events, which means that the probability for any of these events to occur does not depend on previous faults.

Note that these assumptions are also valid for other hardware accelerators, including GPUs, TPUs, NPUs, and AI Engines. This is because all of these hardware accelerators work

the same way, with sequential execution of layers (assumption 1), being mapped to limited hardware resources (assumption 3), and exposed to low fluxes (assumption 2), meaning only one fault type can happen at once (assumption 4).

We use these premises to estimate the FIT rates of a model. First, premise 3 tells us that the **hardware utilization of a layer**, S_{layer}, is a ratio in the interval $(0, 1]$, which can be computed with Equation 1:

$$S_{\text{layer}} = \frac{\min(\#OPs_{\text{layer}},\ \#OPs_{\text{TPU}})}{\#OPs_{\text{TPU}}} \tag{1}$$

where $\#OPs_{\text{layer}}$ refers to the number of multiply-accumulate (MAC) operations in that layer, and $\#OPs_{\text{TPU}}$ is the number of MAC operations that the TPU can handle simultaneously. These values can be obtained by analyzing each layer of a given ML model, or estimated by the layer type, input size, and model weights (such as the kernel size, or the size of the dense layer). Other operations (such as transpose or reshape) have negligible effect, as they are less frequent and less computationally-intensive, and are hence not considered.

Next, to predict the **FIT rate of a fault type in a layer**, as shown in Equation 2, we consider (i) $\text{SDC}_{\text{rate}}^{(f,\,\text{layer})}$, the probability for the fault type in that layer to propagate till the output, (ii) the FIT rate for that fault type FIT_f (according to our fault model presented in Section V-B), and the hardware utilization S_{layer} of the layer (computed with Equation 1).

$$\text{FIT}_{\text{layer}}^{(f)} = \text{SDC}_{\text{rate}}^{(f,\,\text{layer})} \cdot \text{FIT}_f \cdot S_{\text{layer}} \tag{2}$$

After computing the FIT rates for each fault type in a layer, using assumption 4, we can estimate the **full FIT rate of the layer** by adding the partial FIT rates (computed with Equation 2). Finally, according to assumptions 1 and 2, we can calculate the partial FIT rate of each layer separately (as they are processed one at a time), and then combine all layers to estimate the full model FIT rate, as shown in Equation 3:

$$\text{FIT}_{\text{model}} = \frac{1}{\text{N}_{\text{layers}}} \sum_{\text{layer}} \sum_{f \in \{\text{single, small, medium}\}} \text{FIT}_{\text{layer}}^{(f)} \tag{3}$$

VII. Fault Injection Results

In this section, we utilize TPU-FI to answer the following four research questions (RQs):

- **RQ1:** How accurate is TPU-FI in predicting the FIT rate of ML models when compared to beam experiments?
- **RQ2:** How does each fault type influence the overall FIT rate and SDC behavior of the model?
- **RQ3:** How do the structural characteristics of ML models, such as architecture, layer type, and layer position, affect their resilience to faults?
- **RQ4:** How do input characteristics influence fault propagation and model resilience?

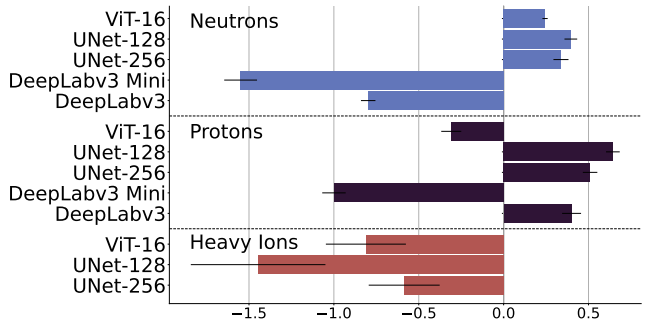


Fig. 6. Difference between the FIT rates estimated with FI and those measured with beam experiments. A negative value means that the FI over-estimated the FIT rate.

A. RQ1: Comparison with Beam Experiments

To assess the accuracy of TPU-FI, we compare the predicted FIT rates via FI to the values obtained from real beam experiments. By employing the methodology presented in Section VI-B, we estimate the FIT rates for the ML models[5], and compare them to the FIT rates measured with accelerated beam experiments[6] (recall that we execute the same code and inputs in both FI and beam experiments).

As shown in Figure 6, across every model tested, our estimates are, on average, within $1.7\times$ of the FIT rates measured with neutron beam experiments ($2.6\times$ in the worst case). The results with heavy ions and protons are similar (within $2.0\times$), confirming the overall accuracy of our cross-layer methodology. *These results are a significant improvement over the state-of-the-art, which can show differences of up to* $10\times$ *when comparing FIT rates obtained with beam and FI [11]*, even with much simpler devices and codes compared to our ML accelerator - in other words, we have improved the accuracy of predicted FIT rates by $\approx 4\times$ for ML accelerators.

Note that differences between the FIT rates estimated with our tool and the measurements obtained via beam experiments are to be expected, as there is a large difference between the methodologies (testing the hardware with radiation Vs simulating the fault in software). Our approach significantly improves the accuracy of the estimated FIT rates for ML models running on the Coral Edge TPU, elevating software-level FI, when combined with a realistic fault model, as a cost- and time-efficient proxy for expensive radiation experiments.

Observation III

TPU-FI estimates FIT rates that are within $1.7\times$ on average of beam results ($2.6\times$ in the worst case), enabling accurate reliability analysis even without beam access.

B. RQ2: Effect of Fault Type

*1) **Comparison of Beam-driven Fault Types**:* Figure 7 shows the probability (measured with FI) for each fault type (single, small-box, and medium-box) to propagate in software

[5]The error bars for fault injection results were computed according to [46].

[6]We follow the methodology described in [7] to compute the error bars for accelerated beam experiments.

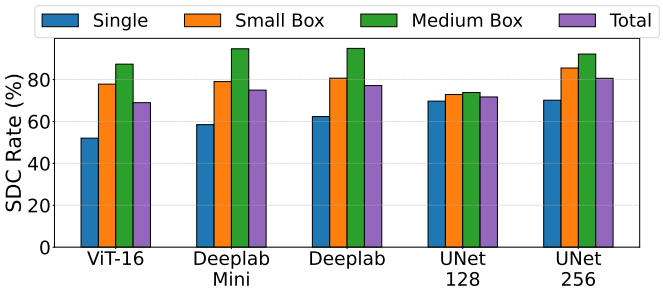


Fig. 7. Comparison of SDC rates (%) across models obtained by TPU-FI. The bars represent the SDC rate under different fault models: single-element, small-box, medium-box, and the weighted total.

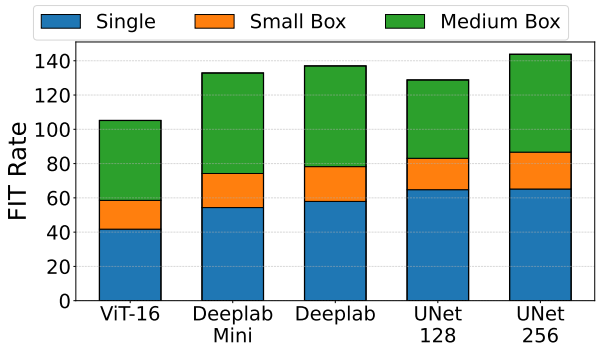


Fig. 8. Comparison of FIT rates across different models. Each bar is segmented by fault type, showing its proportional impact on the FIT rate.

leading to an SDC in the model output. Because medium-box faults affect more elements than the other fault types, they have the highest SDC rates across all models, ranging from 73.8% (UNet-128) to 94.9% (DeepLab). Small-box faults follow, with slightly lower SDC rates (between 70% and 86%), while single-point faults have drastically lower SDC rates, ranging from 45.3% to 70.1%. Figure 8 shows the SDC rate of the entire model, computed by multiplying the probability for a fault type to propagate to the output (Figure 7) by the probability of each fault type to occur (measured with beam experiments). The values range between 64.6% and 80.6%, with ViT being the most resilient and UNet256 being the least.

Further, the trend in the FIT rates for each fault type in Figure 8 is different from the SDC rates in Figure 7, due to the *probability* of occurrence of each fault type. While single-point faults occur most frequently (about 48% of the time), the higher severity of medium-box faults (37% occurrence) means that the **medium-box faults contributes more to the model's FIT rate**. On the other hand, small-box faults, despite having higher SDC rates than single-point faults (Figure 7), have consistently lower FIT rates due to their low probability of occurrence ($\approx$15% in Figure 8). In other words, despite being less likely to propagate to the output (and causing an SDC), single-point faults contribute more to the FIT rate of models due to their higher probabilities of occurrence compared to small-box faults. Finally, medium-box faults also show higher *critical* SDC rates than single-point faults across all models, and their critical FIT rates exceed those of single-point faults in all cases. This is because, as the number of incorrect elements in the output increases, the impact of an error in the overall prediction accuracy of the model also increases.

Observation IV

The FIT rate of a fault type ($\text{FIT}^{(f)}$) depends on two main factors: the severity of that fault type $\text{SDC}_{\text{rate}}^{(f)}$, as well as the probability of occurrence for that fault type $\mathcal{P}(f)$.

*2) **Comparison to Single Bit-Flip Fault Model**:* Traditional software-level FI tools such as TensorFI [47], and PyTorchFI [48] are designed for CPUs, and rely on a simplified fault model: a single-bit flip at a randomly selected output element, with uniform sampling of the bit index. However, these approaches ignore hardware-specific fault characteristics, and, as we have seen from the accelerated beam experiments, this fault model is not accurate for ML accelerators like TPUs.

To quantify this gap, we implemented using TPU-FI, a single-bit fault model (with uniform sampling), and compared it with our beam-driven methodology. Recall that our beam-driven approach, as mentioned in Section V-B, uses a realistic mix of single-point, small-box, and medium-box faults, and samples bit flips from the empirical distribution $\mathcal{D}_{\text{bit}}$ (Table I).

We find that, on average, our method produces 1.32$\times$ higher *critical* SDC rates than the uniform single bit-flip model, with the difference increasing to 1.78$\times$ in the worst case observed. This difference is primarily because the single bit-flip model does not capture the severity of medium-box and small-box fault types. For instance, in DeepLabv3, the critical SDC rate of medium-box faults is 3.3$\times$ higher than the critical SDC rate predicted by the single bit-flip model. *Thus, the uniform single bit-flip model is not accurate for ML accelerators.*

Observation V

Traditional single-bit FI underestimates the impact of faults in ML accelerators. Accurate reliability analysis requires hardware-aware fault modeling that reflects realistic spatial and bit-level error distributions from beam experiments.

C. RQ3: Analysis of algorithmic characteristics

*1) **Impact of ML model Architecture on Resilience**:* We find that, within the same architecture, total SDC rates are

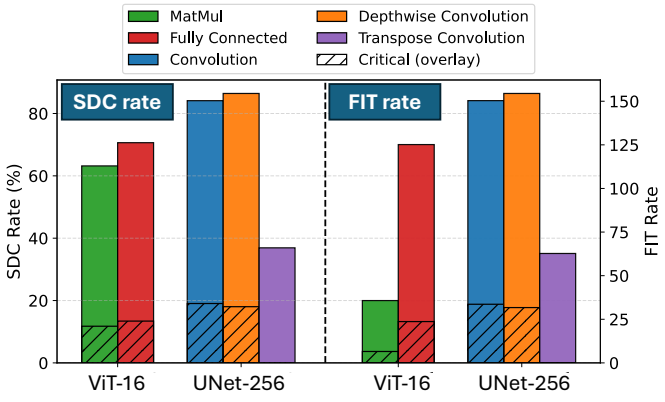


Fig. 9. FIT/SDC rates by layer type in ViT and UNet.

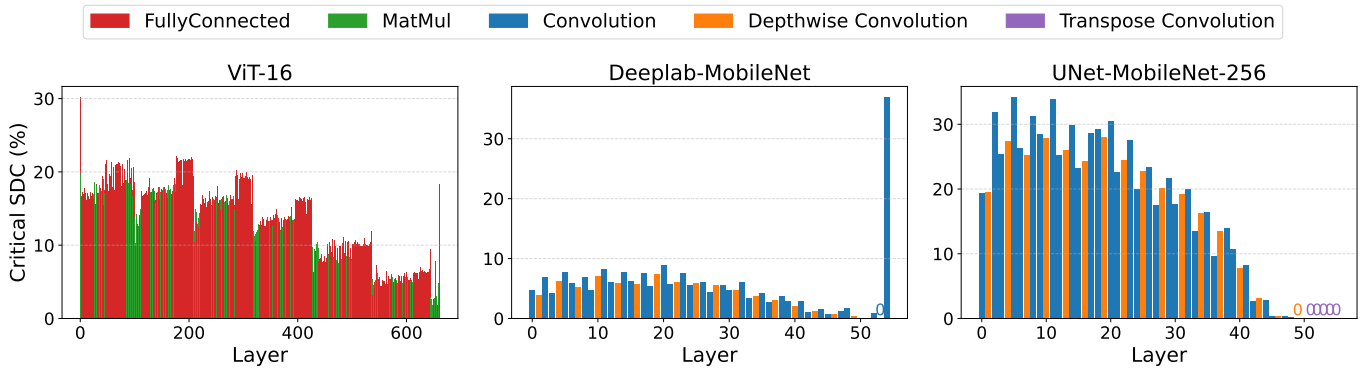


Fig. 10. Critical SDC rate for each layer in the different model architectures we consider.

generally higher for deeper, wider, or larger-input models, while critical SDC rates tend to increase with larger input sizes in UNets. For DeepLab, the only architectural difference between the standard and Mini versions is that the number of filters is doubled in DeepLab (2×). This means that, in DeepLab-Mini, faults affect a larger fraction of the output channels (1 out of 8) compared to DeepLab (1 out of 16). In UNet models, however, the fraction of affected output channels per layer is similar between both UNet models, which explains why DeepLab-Mini has a higher critical SDC rate than DeepLab, while in UNets the effect is less pronounced. Nevertheless, total SDC rates remain higher in the standard DeepLab, consistent with the trends seen in other architectures.

Observation VI

Models sharing the same architectural structure exhibit similar fault sensitivity, but increasing input size or channel width can raise FIT without changing SDC rates.

*2) **Impact of Layer Characteristics on Model Resilience**:* Figure 9 shows FIT and critical FIT per layer type. The clearest pattern is in ViT-16, where fully connected (FC) layers dominate. It is worth noting that, due to the systolic array architecture, large matrix multiplications are instead implemented as FC layers. Therefore, FC layers represent most of the computation in ViT-16, as these layers are used for patch embedding, attention projection, and the final classification head, all of which have a direct influence on model predictions. In contrast, the other MatMul layers (in green) are primarily used within normalization, and scaling steps, making them have less of an effect on the final output.

For CNNs, two clear patterns are observed: 2D, and 3D (depth-wise) convolutions layers are broadly similar, except that 2D convolutional layers show slightly higher *critical* FIT and SDC rates, while 3D layers have slightly higher total FIT and SDC rates. More notable, however, are the transpose convolution layers used in the final decoder stages of UNet models: despite moderate total FIT rates (e.g., 0.047 in UNet-256), they exhibit **zero critical FIT in both UNet models**. This is because transpose convolutions affect only localized pixel regions, limiting their potential to cause misclassifications in the overall prediction.

Observation VII

Fully connected and convolution layers dominate critical fault impact, while transpose convolutions (despite producing non-critical changes) have zero critical FIT.

Figure 10 shows the critical SDC rate for each layer in ViT-16, DeepLab, and UNet-256. Across all models, we observe that layer location plays an important role in fault propagation and severity, with both early and late layers often contributing more significantly to critical SDCs.

In ViT, the highest critical SDC rates appear at the two endpoints: the first layer (patch embedding) with 30.2%, and the final layer (classification head) with 18.4%. ViT consists of six transformer layers, each showing decreasing sensitivity: the average critical SDC rate decreases steadily from 18%, all the way to 5%. This suggests that faults introduced earlier in the model can propagate more widely, affecting subsequent attention heads. The final classification head is highly vulnerable, as bit-flips can directly alter the predicted output.

In DeepLab, critical SDC rates gradually increase up to layer 20 (peaking at 10.3%), then slightly decline. The final layer, which performs per-pixel classification for segmentation, shows the highest vulnerability at 37.4%, aligning with its role in directly producing the output mask. This trend reflects the cumulative nature of feature extraction and the final layer's sensitivity to localized disruptions.

UNet-256 shows a clear trend tied to its encoder-decoder structure: in the downsampling path, the first two layers already reach critical SDC rates around 19%, with later convolutional layers peaking above 33%. After this peak, rates gradually decline through the mid-section of the network, and continue dropping in the upsampling path. Notably, the last four layers, composed of transpose convolutions, exhibit zero critical SDCs, despite showing moderate non-critical effects in other metrics. This reflects the spatially-localized role of these layers, as faults in these layers affect limited regions of the output, and thus do not lead to critical SDCs.

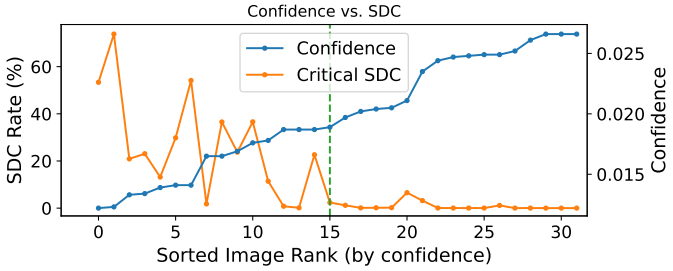


Fig. 11. Confidence of model prediction Vs. its sensitivity to faults (critical SDCs) for different input images for ViT-16.

Observation VIII

Layer sensitivity to faults strongly depends on model structure: early layers amplify fault propagation, while output layers (particularly in transformers and CNN classifiers) directly determine fault severity. In contrast, transpose convolution layers in UNet consistently yield zero critical SDCs, despite non-critical perturbations.

D. RQ4: Input-based Fault Propagation

Fault propagation in ML models can be influenced not only by architecture and fault location, but also by the specific input processed. We explore how input-dependent behavior affects the likelihood and spread of faults in ViT models.

*1) **Confidence Correlation with Fault Robustness**:* We investigate whether the model's confidence in its prediction correlates with fault resilience. We define the *confidence* of a prediction as the difference in probabilities between the two classes with highest scores. For instance, a prediction that assigns 50% probability of class A and 40% of class B has a low confidence, as the difference is only 10%.

Figure 11 shows the sorted per-image confidence values (blue), along with the corresponding critical SDC counts (orange) for ViT-16. We observe a clear trend: inputs with higher confidence experience lower critical SDC rates. Additionally, our analysis reveals that the difference can be extreme: high-confidence inputs present nearly 0% misclassification rate, while other inputs present up to 70% probability of a critical SDC. Thus, predictions made with high certainty are more robust to perturbations due to a larger margin between the correct class and other logits. For non-critical SDCs, which involve minor changes that do not flip the predicted label, we observed that the correlation is weaker, as expected.

*2) **Input-dependent Attention Head Sensitivity**:* We further explore input-dependent fault propagation by analyzing how faults propagate from elsewhere in the network to specific attention heads in transformer block 2 of ViT. Figure 12 shows (a) the total magnitude of the error in each head's output (measured as $\sum|\Delta|$), averaged over every input. While the average perturbation is fairly uniform across heads, the spread is substantial, with some heads showing over 110% variation between the affected inputs. This high variance is because *fault propagation is highly input-dependent*.

Figure 12 also reports (b) the number of inputs for which each head in Block 2 experienced the highest $\sum|\Delta|$ across

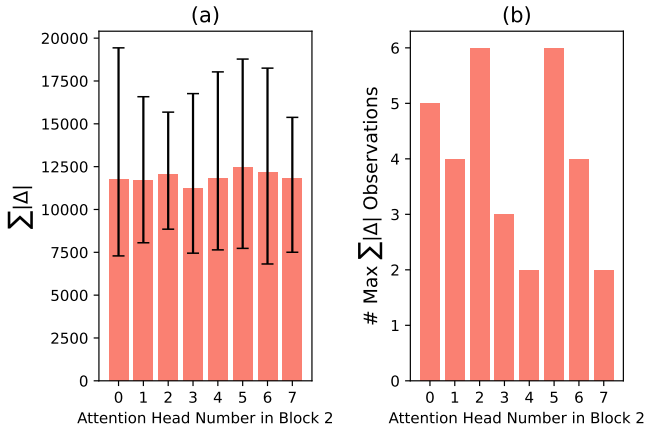


Fig. 12. A) Sum of the differences in the attention heads in Block 2, caused by earlier FIs. Error bars indicate the minimum and maximum values. B) Number of input images for which each head had the highest sum of differences.

all heads, that is, for each attention head (X axis), how many input images affected it the most (Y axis). These results clearly show that different inputs activate different attention heads, creating diverse fault sensitivity profiles even within the same transformer block. Interestingly, for inputs that affected head 5 in block 2, head 0 in the first transformer block (Block 0) was the most impactful injection site. In 4 out of 6 cases, injecting at head 0 of block 0 affected head 2 of block 5, suggesting the presence of a stable inter-block attention pathway influenced by specific inputs. This points to structured, input-driven fault propagation through attention layers.

Observation IX

Inputs significantly influence fault outcomes: confident predictions are more robust to critical SDCs, and different inputs activate distinct attention heads across models.

VIII. Discussion

This section discusses how to generalize our results to other devices and models, and implications for model hardening.

A. Generalization of results

Hardware accelerators: The Google Coral TPU is based on a systolic array architecture, which can also be found in several other COTS ML accelerators, such as Tensor Cores in GPUs [10], and AI Engines in AMD Versal ACAPs [3], [8]. The methodology employed in this work can be easily applied to other accelerators, as the assumptions in Section VI-B apply to most (if not all) accelerators (including GPUs, TPUs, NPUs, and AI Engines). The only missing requirement is to obtain the specific fault model (fault types and distributions) for a given accelerator, following the methodology described in V-B. That is, by performing accelerated beam experiments with convolutions (or matrix multiplications) that saturate the hardware resources in new devices, one can derive the fault model to be used to estimate the FIT rate of full ML models *without requiring to test those models* with beam experiments.

Data types: Although the Coral TPU quantizes models to int8, a distribution of single- and multiple-element errors has been observed in devices using floating-point 16-bit (fp16) or 32-bit (fp32) numbers [13], [49], [50]. We observed erroneous output values that were physically distributed because the int8 quantization in the TPU masks small numerical errors. On the other hand, the literature shows that GPUs running fp32 often show line- or rectangular-shaped errors [50], [51]. Thus, higher precision data types (such as fp32) rarely mask errors. In other words, while the data type used in an ML accelerator does not affect whether errors manifest as single- or multiple-element errors, lower precision reduces the *density* of "bounding boxes" (fewer elements are wrong, as many small errors are masked). Conversely, increasing the data precision also increases the error density, as small numerical errors propagate to the output, as shown in Santos et al. [52].

ML models: While we evaluate TPU-FI with ViTs and semantic segmentation CNNs, matrix multiplications and convolutions (also implemented as matrix multiplications) constitute the bulk of most modern ML models. Thus, TPU-FI (and our entire methodology) apply to any feed-forward ML model, including Large Language Models (LLMs).

ECC: The Google Coral TPU does not have error detection or correction mechanisms including ECC. Some AI accelerators include ECC that has been shown to reduce the FIT rate of ML models, but does not reduce the criticality of errors [51]. In fact, ECC reduces the *probability* of certain error types, but not their *characteristics*. For instance, a radiation-induced error in the input data will likely manifest as either a single-element error, or in a very small region of the layer output of a convolution (due to the small kernel sizes). Thus, adding ECC to protect the input buffer would reduce the probability of occurrence of these types of errors (without affecting their characteristics), but would have little to no effect on medium or large boxes. Conversely, adding ECC to the model parameters would significantly reduce the probability of medium or large boxes, but would have little effect on single-element errors.

B. Impact of results

Layer characteristics: As discussed in Section VII-C, both the characteristics and location of layers affect its criticality. This data can be used to identify layers that are more likely to generate mispredictions. For instance, the earlier layers of a ViT model are more sensitive, a trend that is also seen in UNet-MobileNet-256 (Figure 11). Overall, the trend shows that layers that deal with unprocessed (or less processed) data are more sensitive to errors. This is largely because the output of the earlier layers is used by consequent layers, thus early errors in "understanding" the input can cascade into mispredictions. However, the best way to evaluate the sensitivity of a layer is via fault injection, which can be done using TPU-FI.

Hardening models: Our results identifying the most sensitive layers (Section VII-C) can be leveraged to improve the hardening of ML models. For instance, we can estimate that applying selective triplication (without requiring an oracle) to the last layer of Deeplab-MobileNet (Figure 10) would reduce critical FIT rate by 13% with only a 0.43% performance overhead, compared to the 200% overhead of full triplication. Thus, TPU-FI can be used to identify the critical layers of a given ML model, which can then be used to harden the model with minimal overhead. Other approaches to selectively improve the reliability of ML models include value clipping, checksums, and filters [15], [17], [30], [44], [53], [54].

IX. Conclusion

We implement and evaluate TPU-FI, a software-level FI framework for radiation-induced faults based on realistic data gathered with accelerated beam experiments. Our framework provides FIT rate estimates that are, on average, within $1.7\times$ of the FIT rates measured with beam experiments. TPU-FI provides visibility into how faults propagate in an ML model, revealing insights that would not be accurately captured by traditional FI tools due to their simplified fault models, nor obtained with beam experiments due to their limited visibility.

Consequently, by utilizing TPU-FI, researchers can obtain realistic FIT rates for a given model, *without performing expensive accelerated beam experiments*. Furthermore, TPU-FI faithfully simulates the impact of radiation in ML models deployed on systolic arrays, enabling intricate studies of the reliability of each part of a model architecture. We find that faults caused by radiation in systolic arrays often affect *multiple output elements* in a layer, and that, in each element, the bit-flip distribution is *not uniform*, but rather fall within a bounded box. Our results indicate that single-element faults can under-report misclassification rates by up to $4\times$ compared to faults affecting a large number of elements.

By leveraging TPU-FI, one can improve the reliability of ML models, e.g., understanding the vulnerable blocks of a model allows adding partial redundancy with minimal overhead. Additionally, TPU-FI can be used as an initial validation (with practically no cost) of software-level reliability improvement techniques for ML models.

Acknowledgement

We would like to thank the anonymous DSN reviewers for their hard work, and for the constructive feedback provided. This work was supported in part by ESA's OSIP 4-40005, by UKRI's co-sponsor PhD, by the embassy of Canada in Italy for the innovation award 2024, by the European Union's 2020 research and innovation programme under grant agreement No 101008126 (RADNEXT), by Natural Sciences and Engineering Research Council of Canada (NSERC), and by research gifts from Meta and Google. Neutron beam time was provided by ChipIR (DOI: 10.5286/ISIS.E.RB2500039), and by TRIUMF. Proton beam time was provided by CNAO, and by TIFPA. Ion beam time was provided by RADEF, and by UCL.

REFERENCES

[1] N. P. Jouppi, D. Hyun Yoon, M. Ashcraft, M. Gottscho, T. B. Jablin, G. Kurian, J. Laudon, S. Li, P. Ma, X. Ma, T. Norrie, N. Patil, S. Prasad, C. Young, Z. Zhou, and D. Patterson, "Ten lessons from three generations shaped google's tpuv4i : Industrial product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, Virtual Event, Spain, Jun. 2021, pp. 1–14.

[2] J. Timpe, K. O'Neill, D. Qendri, B. Berkane, G. Chapman, and D. Quinn, "Application of amd versal™ adaptive soc to radar space time adaptive processing in space," in *2023 European Data Handling & Data Processing Conference (EDHPC)*, 2023, pp. 1–5.

[3] P. Maillard, A. Dhavlle, Y. P. Chen, N. Fraser, Y. Chen, N. Vacirca, J. Barton, M. L. Voogel, and M. Sawant, "Protons evaluation of 7nm versal™ ai engine (aie) based radiation tolerant platform for deep learning applications," in *2024 IEEE Radiation Effects Data Workshop (REDW) (in conjunction with 2024 NSREC)*, vol. 1, Ottawa, Canada, Jul. 2024, pp. 89–96.

[4] R. Fan, W. Wang, and X. Chu, "Dtc-spmm: Bridging the gap in accelerating general sparse matrix multiplication with tensor cores," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 253–267. [Online]. Available: https://doi.org/10.1145/3620666.3651378

[5] G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "Silent data corruptions: The stealthy saboteurs of digital integrity," in *2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, 2023, pp. 1–7.

[6] T. Macieira, S. Gurumurthy, S. Gurumurthi, A. Haggag, G. Papadimitriou, and D. Gizopoulos, "Silent data corruptions in computing: Understand and quantify," in *2024 IEEE 30th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, 2024, pp. 1–7.

[7] H. Quinn, "Challenges in testing complex systems," *IEEE Transactions on Nuclear Science*, vol. 61, no. 2, pp. 766–786, Apr. 2014.

[8] H. Quinn, C. Corley, and P. Thelen, "Neutron-induced sees in the xilinx versal prime," in *2022 IEEE Radiation Effects Data Workshop (REDW) (in conjunction with 2022 NSREC)*, vol. 1, Provo, USA, Jul. 2022, pp. 1–8.

[9] M. C. Casey, J. S. Goodwill, E. J. Wyrwas, R. A. Austin, C. M. Wilson, S. D. Stansberry, N. Gorius, and S. Aslam, "Single-event effects on commercial-off-the-shelf edge-processing artificial intelligence asics," *IEEE Transactions on Nuclear Science*, vol. 70, no. 8, pp. 1716–1723, Aug. 2023.

[10] P. Rech, "Artificial neural networks for space and safety-critical applications: Reliability issues and potential solutions," *IEEE Transactions on Nuclear Science*, vol. 71, no. 4, pp. 377–404, Apr. 2024.

[11] P. R. Bodmann, G. Papadimitriou, R. L. R. Junior, D. Gizopoulos, and P. Rech, "Soft error effects on arm microprocessors: Early estimations versus chip measurements," *IEEE Transactions on Computers*, vol. 71, no. 10, pp. 2358–2369, 2022.

[12] J. D. Guerrero Balaguera, J. E. Rodriguez Condia, F. Fernandes Dos Santos, M. Sonza Reorda, and P. Rech, "Understanding the effects of permanent faults in gpu's parallelism management and control units," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '23. New York, NY, USA: Association for Computing Machinery, 2023. [Online]. Available: https://doi.org/10.1145/3581784.3607086

[13] P. Rech, L. Pilla, P. Navaux, and L. Carro, "Impact of gpus parallelism management on safety-critical and hpc applications reliability," in *2014 44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, 2014, pp. 455–466.

[14] O. Chatzopoulos, G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "From gates to SDCs: Understanding fault propagation through the compute stack," in *2025 Design, Automation & Test in Europe Conference (DATE)*, 2025, pp. 1–7.

[15] K. Ma, C. Amarnath, and A. Chatterjee, "Error resilient transformers: A novel soft error vulnerability guided approach to error checking and suppression," in *2023 IEEE European Test Symposium (ETS)*, Venezia, Italy, May 2023, pp. 32–37.

[16] X. Xue, C. Liu, Y. Wang, B. Yang, T. Luo, L. Zhang, H. Li, and X. Li, "Soft error reliability analysis of vision transformers," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 31, no. 12, pp. 2126–2136, Dec. 2023.

[17] G. Gavarini, A. Ruospo, and E. Sanchez, "Evaluation and mitigation of faults affecting swin transformers," in *2023 IEEE 29th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, Crete, Greece, Sep. 2023, pp. 168–174.

[18] O. Chatzopoulos, N. Karystinos, G. Papadimitriou, D. Gizopoulos, H. D. Dixit, and S. Sankar, "Veritas – demystifying silent data corruptions: μarch-level modeling and fleet data of modern x86 cpus," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, 2025, pp. 1–14.

[19] L. Yang, G. Papadimitriou, D. Sartzetakis, A. Jog, E. Smirni, and D. Gizopoulos, "Gpu reliability assessment: Insights across the abstraction layers," in *2024 IEEE International Conference on Cluster Computing (CLUSTER)*, 2024, pp. 1–13.

[20] J. E. R. Condia and M. S. Reorda, "Testing permanent faults in pipeline registers of gpgpus: A multi-kernel approach," in *2019 IEEE 25th International Symposium on On-Line Testing and Robust System Design (IOLTS)*, 2019, pp. 97–102.

[21] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, Jun. 2016, pp. 770–778.

[22] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, Vienna, Austria, May 2021.

[23] Q-Engineering. Google coral edge tpu explained in depth. Accessed: 2023-02-01. [Online]. Available: https://qengineering.eu/google-corals-tpu-explained.html

[24] C. Baraniuk. (2025, Dec.) Bit flips: How cosmic rays grounded a fleet of aircraft. Accessed: 2025-12-02. [Online]. Available: https://www.bbc.com/future/article/20251201-how-cosmic-rays-grounded-thousands-of-aircraft

[25] B. Sangchoolie, K. Pattabiraman, and J. Karlsson, "One bit is (not) enough: An empirical study of the impact of single and multiple bit-flip errors," in *2017 47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2017, pp. 97–108.

[26] C. Bolchini, L. Cassano, A. Miele, and A. Toschi, "Fast and accurate error simulation for cnns against soft errors," *IEEE Transactions on Computers*, vol. 72, no. 4, p. 984–997, Apr. 2023. [Online]. Available: http://dx.doi.org/10.1109/TC.2022.3184274

[27] F. F. d. Santos, J. E. R. Condia, L. Carro, M. S. Reorda, and P. Rech, "Revealing gpus vulnerabilities by combining register-transfer and software-level fault injection," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2021, pp. 292–304.

[28] A. Tyagi, Y. Gan, S. Liu, B. Yu, P. Whatmough, and Y. Zhu, "Thales: Formulating and estimating architectural vulnerability factors for dnn accelerators," *arXiv preprint arXiv:2212.02649*, 2022.

[29] Y. He, P. Balaprakash, and Y. Li, "Fidelity: Efficient resilience analysis framework for deep learning accelerators," in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2020, pp. 270–281.

[30] Y. He, M. Hutton, S. Chan, R. De Gruijl, R. Govindaraju, N. Patil, and Y. Li, "Understanding and mitigating hardware failures in deep learning training systems," in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–16.

[31] O. Ronneberger, P. Fischer, and T. Brox, "U-net: Convolutional networks for biomedical image segmentation," *arXiv preprint 1505:04597*, May 2015.

[32] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint 1704:04861*, Apr. 2017.

[33] L.-C. Chen, G. Papandreou, F. Schroff, and H. Adam, "Rethinking atrous convolution for semantic image segmentation," 2017. [Online]. Available: https://arxiv.org/abs/1706.05587

[34] B. C. Reidy, M. Mohammadi, M. E. Elbtity, and R. Zand, "Efficient deployment of transformer models on edge TPU accelerators: A real system evaluation," in *Architecture and System Support for Transformer Models (ASSYST @ISCA 2023)*, 2023. [Online]. Available: https://openreview.net/forum?id=PibYaG2C7An

[35] M. Cordts, M. Omran, S. Ramos, T. Rehfeld, M. Enzweiler, R. Benenson, U. Franke, S. Roth, and B. Schiele, "The cityscapes dataset for

semantic urban scene understanding," in *Proc. of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[36] O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, "Cats and dogs," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2012.

[37] A. Krizhevsky, V. Nair, and G. Hinton, "Cifar-100 (canadian institute for advanced research)." [Online]. Available: http://www.cs.toronto.edu/~kriz/cifar.html

[38] M. Everingham, L. V. Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The pascal visual object classes (voc) challenge," 2010.

[39] JEDEC, "Measurement and Reporting of Alpha Particle and Terrestrial Cosmic Ray-Induced Soft Errors in Semiconductor Devices," JEDEC Standard, Tech. Rep. JESD89A, 2006.

[40] *Road vehicles — Functional safety*, International Organization for Standardization ISO 26 262, Dec. 2018.

[41] *Particle and UV radiation testing for space materials*, European Cooperation for Space Standardization ECSS ECSS-Q-ST-70-06C, Jul. 2008.

[42] T. Oldham and F. McLean, "Total ionizing dose effects in mos oxides and devices," *IEEE Transactions on Nuclear Science*, vol. 50, no. 3, pp. 483–499, 2003.

[43] R. L. R. Junior and P. Rech, "Reliability of google's tensor processing units for convolutional neural networks," in *2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks - Supplemental Volume (DSN-S)*, 2022, pp. 25–27.

[44] J. Kosaian and K. V. Rashmi, "Arithmetic-intensity-guided fault tolerance for neural network inference on gpus," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: https://doi.org/10.1145/3458817.3476184

[45] A. Ruospo, M. S. Reorda, R. Mariani, and E. Sanchez, "An effective iterative statistical fault injection methodology for deep neural networks," *IEEE Transactions on Computers*, vol. 74, no. 7, pp. 2431–2444, 2025.

[46] S. K. S. Hari, T. Tsai, M. Stephenson, S. W. Keckler, and J. Emer, "Sassifi: An architecture-level fault injection tool for gpu application resilience evaluation," in *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2017, pp. 249–258.

[47] G. Li, K. Pattabiraman, and N. DeBardeleben, "Tensorfi: A configurable fault injector for tensorflow applications," in *2018 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, 2018, pp. 313–320.

[48] A. Mahmoud, N. Aggarwal, A. Nobbe, J. R. S. Vicarte, S. V. Adve, C. W. Fletcher, I. Frosio, and S. K. S. Hari, "Pytorchfi: A runtime perturbation tool for dnns," in *2020 50th Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W)*, 2020, pp. 25–31.

[49] P. M. Basso, F. F. d. Santos, and P. Rech, "Impact of tensor cores and mixed precision on the reliability of matrix multiplication in gpus," *IEEE Transactions on Nuclear Science*, vol. 67, no. 7, pp. 1560–1565, 2020.

[50] G. Li, K. Pattabiraman, C.-Y. Cher, and P. Bose, "Understanding error propagation in gpgpu applications," in *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2016, pp. 240–251.

[51] F. F. d. Santos, A. Kritikakou, J. E. R. Condia, J.-D. Guerrero-Balaguera, M. S. Reorda, O. Sentieys, and P. Rech, "Characterizing a neutron-induced fault model for deep neural networks," *IEEE Transactions on Nuclear Science*, vol. 70, no. 4, pp. 370–380, 2023.

[52] F. F. d. Santos, P. F. Pimenta, C. Lunardi, L. Draghetti, L. Carro, D. Kaeli, and P. Rech, "Analyzing and increasing the reliability of convolutional neural networks on gpus," *IEEE Transactions on Reliability*, vol. 68, no. 2, pp. 663–677, 2019.

[53] Z. Chen, G. Li, and K. Pattabiraman, "A low-cost fault corrector for deep neural networks through range restriction," in *2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*. Los Alamitos, CA, USA: IEEE Computer Society, Jun. 2021, pp. 1–13. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/DSN48987.2021.00018

[54] L. Roquet, F. Fernandes dos Santos, P. Rech, M. Traiola, O. Sentieys, and A. Kritikakou, "Cross-Layer Reliability Evaluation and Efficient Hardening of Large Vision Transformers Models," in *Design Automation Conference (DAC) preprint*, San Francisco, CA, USA, Jun. 2024.